

Practical No: - CASE STUDIES

ON

Cryptography and Network Security - SSCS 4020

TITLE: Case Studies on Message Authentication, Hash Functions,
Digital Signatures & DSS

BACHLEOR OF SCIENCE IN INFORMATION TECHNOLOGY
(HONOURS.)

SUBMITTED BY:

Name: Shah Rishabh Alpeshbhai

Enrolment: 23SS02IT181

Class: BSCIT-H – 7A - Batch 2023-27

Case Study 1: Banking Transaction Authentication — MAC & Hash Functions

Q1. Identify the authentication requirements involved in this scenario.

The main authentication and security requirements are:

1. **Data Integrity:** The transaction amount and account details should not be modified while travelling through the network.
2. **Message Authentication:** The bank should verify that the transaction was generated by an authorized system.
3. **Protection Against Modification:** An attacker should not be able to change ₹50,000 to ₹5,00,000 without detection.
4. **Confidentiality:** If required, encryption can be used to keep transaction information secret.
5. **Key Security:** The secret key used for authentication must be protected from attackers.

Therefore, the bank needs a mechanism that can provide **message integrity and authentication**, such as a MAC or HMAC.

Q2. Explain why a simple hash such as SHA-256 alone does not authenticate the sender.

A simple hash function such as **SHA-256** provides a fixed-size digest of a message. It can help detect whether the message has changed.

However, it **does not authenticate the sender** because a hash does not use a secret key.

For example:

Original Message → SHA-256 → Hash

If an attacker changes:

Amount = ₹50,000

to:

Amount = ₹5,00,000

the attacker can calculate a new SHA-256 hash for the modified message.

Therefore, the receiver cannot determine whether the hash was generated by the authorized sender.

A **MAC/HMAC** uses a secret key, so only parties possessing the secret key can generate a valid authentication code.

Q3. Explain how a MAC (Message Authentication Code) can protect the transaction.

A **Message Authentication Code (MAC)** is a security mechanism that uses a secret key to generate an authentication code for a message.

The process is:

Transaction Message + Secret Key

↓

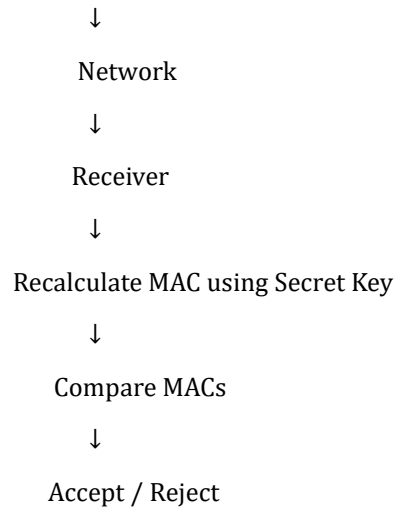
MAC Algorithm

↓

MAC

↓

Message + MAC



If the message is changed during transmission, the receiver calculates a different MAC.

For example:

Original:

Account A → Account B → ₹50,000

Modified:

Account A → Account B → ₹5,00,000

The MAC of the modified message will not match the original MAC. Therefore, the bank rejects the transaction.

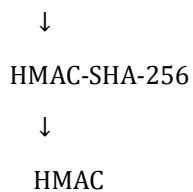
Q4. Explain the role of HMAC-SHA-256 and the secret key.

HMAC-SHA-256 combines a secret key with the SHA-256 hash function to provide message authentication.

The secret key is known only to the authorized parties.

Basic process:

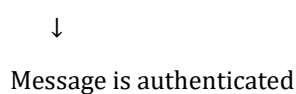
Message + Secret Key



At the receiving side, the same secret key is used to calculate the HMAC again.

If both HMAC values are equal:

Calculated HMAC = Received HMAC



If they are different:

Calculated HMAC ≠ Received HMAC

↓

Message may have been modified

Thus, HMAC-SHA-256 provides **message integrity and authentication**.

Q5. What happens when the transaction amount is modified?

Suppose the original transaction is:

Account A → Account B → ₹50,000

An attacker changes it to:

Account A → Account B → ₹5,00,000

The message content has changed.

Therefore, when the bank recalculates the MAC using the secret key, it produces a different MAC.

Original MAC ≠ New MAC

The bank detects the modification and **rejects the transaction**.

Q6. Why cannot an attacker simply calculate a new valid MAC without the secret key?

A MAC is generated using a **secret key**.

The attacker may know:

- The original message
- The MAC
- The MAC algorithm

But the attacker does not know the secret key.

Without the secret key, the attacker cannot efficiently generate a valid MAC for the modified transaction.

Therefore:

Modified Message + Unknown Secret Key

↓

Cannot generate valid MAC

This prevents unauthorized modification of authenticated messages.

Q7. What security properties does a MAC provide?

A MAC mainly provides:

1. **Message Integrity** – Detects whether a message has been modified.
2. **Message Authentication** – Verifies that the message was generated by someone possessing the shared secret key.
3. **Modification Detection** – Detects unauthorized changes to the message.

A MAC does **not** normally provide confidentiality because it does not encrypt the message.

Q8. Does MAC provide non-repudiation?

No.

A MAC does not provide strong non-repudiation.

This is because the sender and receiver share the same secret key. Either party could potentially generate a valid MAC.

For non-repudiation, a **digital signature** is more suitable because it uses a private signing key and a corresponding public verification key.

Q9. What happens if the shared secret MAC key is compromised?

If an attacker obtains the secret MAC key, the security of the MAC is compromised.

The attacker may be able to:

- Generate valid MACs.
- Modify messages and create new valid MACs.
- Pretend to be an authorized party.

Therefore, the compromised key should be **revoked/replaced**, and a new secure key should be established.

Q10. Why would a bank use HMAC instead of an ordinary hash?

A bank would use **HMAC** because it combines hashing with a secret key.

An ordinary hash:

Message → SHA-256 → Hash

does not authenticate the sender.

HMAC:

Message + Secret Key → HMAC-SHA-256 → Authentication Code

provides both **integrity and message authentication**.

Therefore, HMAC is more suitable for security-sensitive transaction authentication.

Case Study 2: Software Company — MD5, SHA & Birthday Attack

Q11. Explain the purpose of a cryptographic hash function.

A **cryptographic hash function** converts input data of arbitrary size into a fixed-size output called a **hash value, digest, or message digest**.

Example:

Software File

↓

Hash Function

↓

Hash / Digest

The main purposes are:

1. **Integrity checking** – Detect whether data has been changed.
2. **Data verification** – Compare the hash of a downloaded file with the published hash.
3. **Digital signatures** – A hash of the document can be signed instead of signing the complete document.

4. **Password protection** – Hashing can be used as part of secure password-storage systems.

A secure cryptographic hash should make it computationally difficult to find inputs that violate its security properties.

Q12. List the important requirements of a secure hash function.

The three important requirements are:

1. Preimage Resistance

Given a hash value H, it should be computationally difficult to find a message M such that:

$$\text{Hash}(M) = H$$

2. Second-Preimage Resistance

Given a specific message M1, it should be difficult to find a different message M2 such that:

$$\text{Hash}(M1) = \text{Hash}(M2)$$

3. Collision Resistance

It should be difficult to find **any two different messages** that produce the same hash:

$$M1 \neq M2$$

but

$$\text{Hash}(M1) = \text{Hash}(M2)$$

These properties help make a hash function suitable for security applications.

Q13. Explain why MD5 is considered cryptographically broken for collision resistance.

MD5 produces a 128-bit hash value.

MD5 is considered cryptographically broken for **collision resistance** because practical collision attacks have been demonstrated.

A collision means:

$$\text{File A} \neq \text{File B}$$

but

$$\text{MD5}(\text{File A}) = \text{MD5}(\text{File B})$$

An attacker can exploit collision weaknesses to create different data having the same MD5 digest.

Therefore, MD5 should not be used for security-sensitive applications where collision resistance is required.

The assignment specifically identifies MD5 as **"Broken"** for collision security.

Q14. Explain the Birthday Attack in the context of hash collisions.

A **Birthday Attack** is a method used to find collisions in a hash function.

A collision occurs when two different messages produce the same hash value:

$$\text{Message A} \rightarrow \text{Hash} \rightarrow H$$

$$\text{Message B} \rightarrow \text{Hash} \rightarrow H$$

where:

$$\text{Message A} \neq \text{Message B}$$

For an n-bit hash, the generic collision-search complexity is approximately:

$$2^{(n/2)}$$

This is much lower than 2^n , which is why the birthday attack is important when evaluating collision resistance.

The assignment gives the same generic relationship for an n-bit hash.

Q15. Compare MD5, SHA-1, SHA-256, and SHA-512.

Algorithm	Digest Size	Collision Security
MD5	128 bits	Broken
SHA-1	160 bits	Collision attacks demonstrated
SHA-256	256 bits	Widely used
SHA-512	512 bits	Widely used

The assignment provides these digest sizes and collision-security descriptions directly.

In simple terms:

- **MD5:** Not suitable for security-sensitive collision-resistant applications.
- **SHA-1:** Also no longer considered suitable where collision resistance is required.
- **SHA-256:** Widely used for modern security applications.
- **SHA-512:** Provides a larger 512-bit digest and is also widely used.

Q16. Explain why the generic birthday-search complexity for an n-bit hash is approximately $2^{(n/2)}$.

An n-bit hash has:

$$2^n$$

possible hash values.

Because of the **birthday paradox**, a collision becomes likely after approximately the square root of the total number of possible hash values.

Therefore:

$$\sqrt{2^n}$$

can be written as:

$$2^{(n/2)}$$

So, the generic birthday-search complexity for an n-bit hash is approximately:

$$2^{(n/2)}$$

For example, for a 256-bit hash:

$$2^{(256/2)}$$

$$= 2^{128}$$

The assignment explicitly gives this SHA-256 calculation.

Q17. For a 256-bit hash, calculate the approximate generic birthday-attack complexity.

Given:

$n = 256$ bits

Formula:

$2^{(n/2)}$

Substitute $n = 256$:

$2^{(256/2)}$

$= 2^{128}$

Therefore:

Approximate generic birthday-attack complexity = 2^{128} attempts.

This is an extremely large number of attempts, illustrating the strong generic collision resistance of a 256-bit hash

Q18. Why is MD5 unsuitable for security-sensitive digital signatures?

MD5 is unsuitable because its **collision resistance is broken**.

In digital signatures, an attacker should not be able to create two different documents with the same digest.

If collisions can be constructed, the security of systems relying on MD5's collision resistance can be undermined.

Therefore, modern secure hash functions such as SHA-256 are preferred for security-sensitive applications.

Q19. Is a hash function itself an authentication mechanism?

No.

A normal hash function provides a digest that can be used to check data integrity, but it does not prove who generated the message.

For example:

Message $\rightarrow$ SHA-256 $\rightarrow$ Hash

Anyone who changes the message can calculate a new hash.

Authentication requires an additional mechanism such as:

- **MAC/HMAC**, using a shared secret key, or
- **Digital signature**, using public-key cryptography.

Q20. What is the difference between a collision attack and a preimage attack?

Collision Attack

Finds two different messages with the same hash.

Attacker chooses two messages.

Goal: $H(M1) = H(M2)$ where $M1 \neq M2$.

Related to collision resistance.

Preimage Attack

Finds a message corresponding to a given hash.

Hash value is already given.

Goal: find M such that $H(M) = \text{given hash}$.

Related to preimage resistance.

Example of collision:

$M1 \neq M2$

$\text{Hash}(M1) = \text{Hash}(M2)$

Example of preimage:

Given H

Find M such that:

$\text{Hash}(M) = H$

Q21. Why is SHA-256 preferred over MD5 for security-sensitive applications?

SHA-256 is preferred because it has a **256-bit digest** and is widely used, whereas MD5 has a 128-bit digest and its collision resistance is broken.

Comparison:

Feature	MD5	SHA-256
Digest size	128 bits	256 bits
Collision security	Broken	Widely used
Security-sensitive use	Not recommended	Commonly used

Therefore, SHA-256 is a more appropriate choice for security-sensitive applications requiring strong collision resistance.

Case Study 3: Government Document Signing — Digital Signature & DSS

Q22. Explain why the document is hashed before signing.

A document is hashed before signing because the hash produces a **fixed-size message digest**.

For a large document, signing the complete document directly can be inefficient.

The process is:

Original Document

↓

Hash Function

↓

Message Digest

↓

Private Key + DSS

↓

Digital Signature

When the document is later verified, the receiver calculates its hash again.

If the calculated digest matches the digest associated with the valid signature, the document has not been changed.

The assignment's digital-signature flow follows this process.

Q23. Explain the difference between a private key, public key, and digital signature.

Term	Meaning
Private Key	Secret key controlled by the signer and used for creating a digital signature.
Public Key	Key that can be distributed publicly and is used to verify the signature.
Digital Signature	A cryptographic value created using the signer's private key and the document's digest.

Simple flow:

Document

↓

Hash

↓

Digest

↓

Private Key

↓

Digital Signature

Verification uses:

Digital Signature + Public Key

↓

Verification

Q24. Explain how digital signatures provide integrity and authentication, and how they support non-repudiation.

A digital signature provides several security properties.

1. Integrity

If the document is changed after signing, its hash changes.

Therefore, signature verification fails.

Original Document → Hash 1

Modified Document → Hash 2

Hash 1 ≠ Hash 2

2. Authentication

The signature is created using the signer's private key. Verification uses the corresponding public key.

This allows the recipient to verify that the signature corresponds to the claimed signing key.

3. Non-repudiation

A properly implemented digital-signature system can provide evidence associated with a particular signing key and support non-repudiation.

However, in a real dispute, additional factors such as key protection, identity verification, certificates, and system records may also be relevant.

Q25. Explain the role of DSS (Digital Signature Standard).

DSS stands for Digital Signature Standard.

It provides a standardized framework for creating and verifying digital signatures.

In the assignment scenario, DSS is used to digitally sign government documents.

The basic process is:

Document

↓

Hash

↓

Message Digest

↓

DSS + Private Key

↓

Digital Signature

During verification:

Document → Hash → Digest 1

Signature + Public Key

↓

DSS Verification

↓

Digest 2

If the required values match, the signature is considered valid

Q26. What happens if an attacker modifies an amount in the document after it has been signed?

Suppose the original document contains:

Amount = ₹50,000

After signing, an attacker changes it to:

Amount = ₹5,00,000

The document's hash will change.

Therefore:

Original Digest ≠ Modified Document Digest

During verification, the signature will not validate against the modified document.

Therefore, the receiver can detect that the document has been **modified after signing**.

Q27. Why must the private signing key never be shared?

The private signing key must remain secret because it is used to create digital signatures.

If another person obtains the private key, they may be able to create signatures that appear to originate from the legitimate signer.

This can lead to:

- Unauthorized documents being signed.
- False signatures.
- Loss of trust in the signing system.
- Potential disputes about signatures.

Therefore:

Private Key → Keep Secret

Public Key → Can Be Shared

Q28. Compare MAC and Digital Signature in terms of keys, authentication, integrity, non-repudiation, and verification.

Feature	MAC	Digital Signature
Keys	Shared secret key	Private key + public key
Authentication	Yes, among parties sharing the secret	Yes, using the public verification key
Integrity	Yes	Yes
Non-repudiation	No	Supports non-repudiation
Verification	Uses shared secret key	Uses public key
Key sharing	Secret key must be shared between authorized parties	Private key remains secret; public key can be distributed
Main use	Message authentication between trusted parties	Signing documents/messages and public verification

Q29. What evidence does a successfully verified digital signature provide?

A successfully verified digital signature provides evidence that:

1. The signature corresponds to the verification key.
2. The signed document has not been altered in a way that would invalidate the signature.
3. The signature is associated with the corresponding signing key.

However, determining the real-world identity of the signer may require additional information such as certificates and identity-management records.

Q30. What additional factors may be relevant when investigating a dispute over who signed a document?

Several factors may be relevant, including:

1. **Digital certificate** associated with the public key.
2. **Certificate authority records**, where applicable.
3. **Private-key protection** and evidence of who controlled the key.
4. **Timestamp information**.
5. **System and audit logs**.
6. **Identity verification records**.
7. Whether the private key was compromised or misused.

Therefore, a digital signature is important evidence, but real-world disputes may require examination of the complete signing environment.

Q31. Why is a public key suitable for signature verification while the private key must remain secret?

The **public key** is designed to be distributed and allows other people to verify a signature.

The **private key** is used to create the signature and must remain secret.

The basic principle is:

Private Key → Signing

Public Key → Verification

If the private key becomes public, an attacker could potentially create unauthorized signatures.

Therefore:

Public key = Shareable

Private key = Secret